Web and Data Engineering

Vorlesung 07: Sonstiges — Vertiefungen

Prof. Dr. Michael Granitzer

Sonstiges

Diese Einheit sammelt Vertiefungen, die in den bisherigen Vorlesungen zu detailliert gewesen wären, aber im Verständnis moderner Web-Systeme hilfreich sind.

Fokus

- wie JavaScript intern arbeitet (Event Loop, asynchrone Ausführung)
- warum scheinbar einfache Seiten langsam sein können (Rendering- und Ladestrategien)

JavaScript-Laufzeit

Leitfrage: Wie führt der Browser JavaScript aus — und warum ist das Ausführungsmodell (single-threaded, ereignisbasiert) die Ursache fast aller Performance- und Reihenfolge-Überraschungen?

Speaker notes

Diese Vertiefung knüpft an das JavaScript-Kapitel aus Vorlesung 03 an. Dort haben wir nur festgestellt, dass JavaScript single-threaded und asynchron ist. Hier zeigen wir konkret, wie sich das im Code äußert: Welche Zeile läuft wann? Warum ist die Reihenfolge nicht die, die man auf den ersten Blick vermutet? Das Modell hinter der Antwort ist der **Event Loop** — die zentrale Abstraktion hinter jedem modernen JavaScript-Programm, sowohl im Browser als auch in Node.js.

Was gibt dieser Code aus?

```
console.log("1: Start");

setTimeout(() => console.log("2: Timeout"), 0);

fetch('/api/products/42')
  .then(response => response.json())
  .then(data => console.log("3: Daten geladen"));

console.log("4: Ende");
```

Aufgabe: In welcher Reihenfolge erscheinen die vier Zeilen in der Konsole? Begründen Sie.

Speaker notes

Lassen Sie die Studierenden zwei Minuten überlegen, bevor Sie auflösen. Die intuitive (falsche) Erwartung ist meistens, dass `setTimeout(..., 0)` „sofort" ausgeführt wird oder dass `fetch` synchron blockiert. Die richtige Antwort erfordert das Verständnis, dass JavaScript single-threaded ist und alle asynchronen Operationen erst nach dem aktuellen Skript abgearbeitet werden. Wer das hier richtig vorhersagt, hat das Event-Loop-Modell verstanden — und kann die meisten Bug-Berichte über „die Daten kommen zu spät an" einordnen.

Auflösung

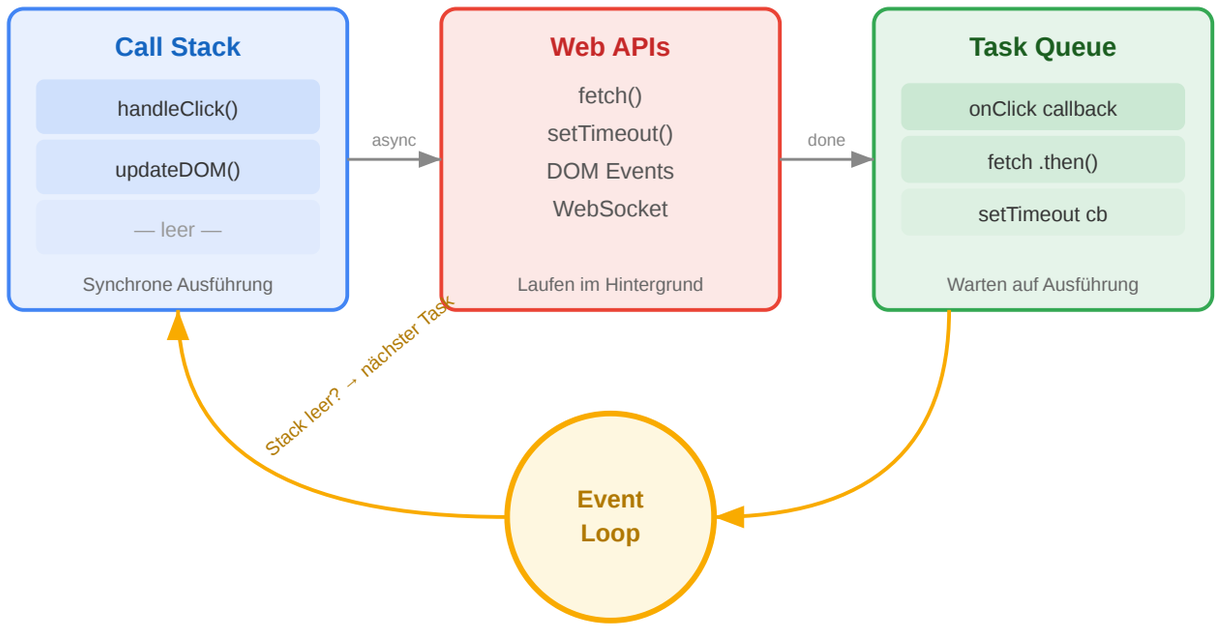
`1: Start → 4: Ende → 2: Timeout → 3: Daten geladen`

JavaScript ist **single-threaded —** aber `setTimeout` und `fetch` laufen im Hintergrund. Der aktuelle Code wird zuerst vollständig ausgeführt, dann kommen die Callbacks. Das ist das fundamentale Ausführungsmodell, das Web-Anwendungen performant macht.

Speaker notes

Wichtig zu betonen: `setTimeout(..., 0)` bedeutet *nicht* „sofort", sondern „so früh wie möglich, nachdem der aktuelle Stack leer ist". Selbst mit Verzögerung 0 wird der Callback in die Task Queue gestellt und dort vom Event Loop abgearbeitet. Das gleiche gilt für Promises (`.then`) und `fetch` — sie kommen aus der Microtask Queue, die vor der Task Queue abgearbeitet wird, weshalb `Promise.resolve().then(...)` in der Praxis vor einem `setTimeout(..., 0)` läuft. Für die Vorlesung reicht das vereinfachte Modell „aktueller Code zuerst, alles andere danach".

Der Event Loop: Warum das Web nicht blockiert



Wie es funktioniert

1. **Call Stack:** Führt den aktuellen Code synchron aus
2. **Web APIs:** Browser übernimmt Netzwerk, Timer, Events im Hintergrund
3. **Task Queue:** Fertige Callbacks warten hier
4. **Event Loop:** Wenn der Call Stack leer ist, holt er den nächsten Callback

Warum das architektonisch wichtig ist

- Ein **einziger Thread** für UI und Logik → blockierende Operationen frieren die Seite ein
- Deshalb ist **alles I/O asynchron** (Netzwerk, Timer, File API)
- Node.js nutzt dasselbe Modell serverseitig → ein Thread bedient tausende Verbindungen

Speaker notes

Die wichtige Konsequenz für die Praxis: Wenn Sie in JavaScript eine Schleife haben, die 500ms läuft, friert die Seite für 500ms komplett ein — keine Klicks, keine Animationen, keine Eingaben. Der Browser kann erst wieder reagieren, wenn der Call Stack leer ist. Lange Berechnungen müssen deshalb entweder in **Web Workers** ausgelagert werden (echter zweiter Thread) oder in kleine Häppchen aufgeteilt und mit `setTimeout` oder `requestIdleCallback` über die Zeit verteilt werden. Dass Node.js dasselbe Modell serverseitig verwendet, ist kein Zufall: Webserver verbringen die meiste Zeit mit Warten auf I/O — ein einziger Thread mit Event Loop kann tausende Verbindungen bedienen, weil er nie blockiert.

Warum ist diese Seite langsam?

Leitfrage: Warum lädt eine scheinbar einfache Seite mehrere Sekunden — und welche Reihenfolge von Ressourcen im <head> entscheidet über den Rendering-Start?

Speaker notes

Dieses Modul vertieft die Rendering-Pipeline aus Vorlesung 03 um einen sehr konkreten Fall: Das Laden von Scripts und Stylesheets. Die Pipeline selbst ist dort erklärt; hier geht es um die *Ladeseite* des Problems — welche Ressourcen blockieren den Parser, welche nicht, und warum die Reihenfolge im HTML-Head den Unterschied zwischen 4 Sekunden und 400 ms ausmachen kann.

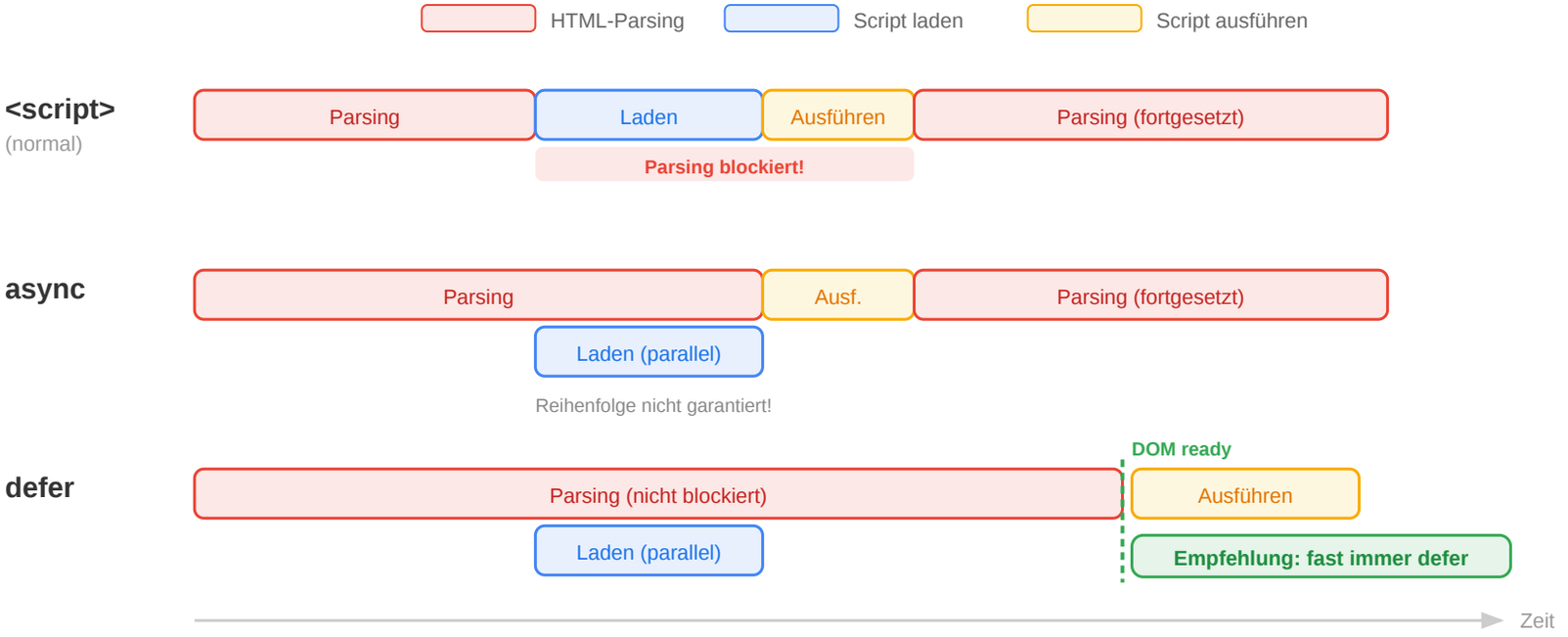
Warum ist diese Seite langsam?

Aufgabe: Eine Produktseite des Online-Shops lädt in 4,2 Sekunden. Der Entwickler hat den HTML-Head so strukturiert:

```
<head>
  <script src="analytics.js"></script>
  <script src="app.js"></script>
  <link rel="stylesheet" href="styles.css">
  <script src="chat-widget.js"></script>
</head>
```

Was ist das Problem — und wie würden Sie es lösen?

Script-Ladestrategien



Speaker notes

Lassen Sie die Studierenden das Problem selbst finden, bevor Sie auflösen. Der springende Punkt: Jedes `<script>` ohne `defer` oder `async` stoppt den HTML-Parser. Der Browser kann nichts rendern, weil das Script potenziell `document.write` aufrufen oder den DOM ändern könnte. Drei Scripts hintereinander bedeuten drei sequenzielle Blockaden. Und weil das CSS *nach* den Scripts kommt, wartet der Browser auf die Scripts, bevor er das Stylesheet überhaupt lädt. Ergebnis: Der Nutzer sieht eine weiße Seite, bis alle Ressourcen geladen sind.

Analyse

Problem	Ursache	Lösung
Scripts blockieren Parsing	`<script>` ohne `defer`/`async` stoppt den HTML-Parser	`<script src="app.js" defer>`
CSS kommt nach Scripts	Browser wartet auf Scripts, bevor er CSS sieht	CSS **vor** Scripts im `<head>`
Alles synchron geladen	3 Scripts + 1 CSS = sequenzielle Blockade	`defer` für alle Scripts, CSS zuerst

Optimierte Version

```
<head>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js" defer></script>
  <script src="analytics.js" defer></script>
  <script src="chat-widget.js" defer></script>
</head>
```

`defer` = Script wird **nach** dem Parsen ausgeführt, **in Reihenfolge**. Das ist fast immer die richtige Wahl.

Speaker notes

Der Unterschied zwischen `defer` und `async`: `defer` lädt parallel zum HTML-Parsing und führt nach dem Parsing in der Reihenfolge des Quelltexts aus — deshalb sicher für Scripts mit Abhängigkeiten. `async` lädt ebenfalls parallel, führt aber sofort beim Eintreffen aus, in unvorhersehbarer Reihenfolge — geeignet für unabhängige Scripts wie Analytics, die keinen anderen Code brauchen. Als Faustregel: `defer` ist die bessere Default-Wahl. `async` nur für wirklich unabhängige, feuer-und-vergiss-Scripts. Eine dritte Alternative ist, Scripts ans Ende des `<body>` zu setzen — funktioniert ebenfalls, ist aber weniger deklarativ und trennt den Head von seiner Konfiguration.

Wrap-Up

- Der **Event Loop** erklärt, warum JavaScript-Code nicht in der Reihenfolge läuft, in der er geschrieben ist: synchroner Code zuerst, Callbacks danach, Microtasks vor Tasks.
- **Ladezeit-Probleme** im Frontend entstehen oft nicht *im* Code, sondern an der Reihenfolge von Ressourcen: render-blocking Scripts, CSS nach Scripts, fehlendes `defer`.
- Beides sind Vertiefungen zum Zusammenspiel der Technologien, das in Vorlesung 03 umrissen wurde.

Speaker notes

Diese Sammel-Vorlesung kann im Laufe des Semesters um weitere Vertiefungen ergänzt werden — etwa zu Web Workers, Service Workers, Caching-Strategien oder zu spezifischen Debugging-Techniken. Der gemeinsame Nenner: Themen, die für das Gesamtverständnis wichtig sind, aber in den regulären Einheiten den roten Faden stören würden.